

⇒ TURING MACHINE:

- It is a general model of computation. The TM can accept a language string as well as compute a function.

- a turing machine is a 5 tuple:

$$T = (Q, \Sigma, \Gamma, q_0, \delta)$$

where,

- Q is a finite set of states
- the 2 halt states h_a, h_r are not elements of Q
- Σ the input symbol
- Γ the tape symbol
- the blank symbol (Δ) is not a subset of Γ
- $q_0 \in Q$
- δ - transition function :

$$\delta : Q \times (\Gamma \cup \{\Delta\}) \rightarrow (Q \cup \{h_a, h_r\}) \times (\Gamma \cup \{\Delta\}) \times \{R, L, S\}$$

7-tuple form.

$$T = (Q, \Sigma, \Gamma, \delta, q_0, A, F)$$

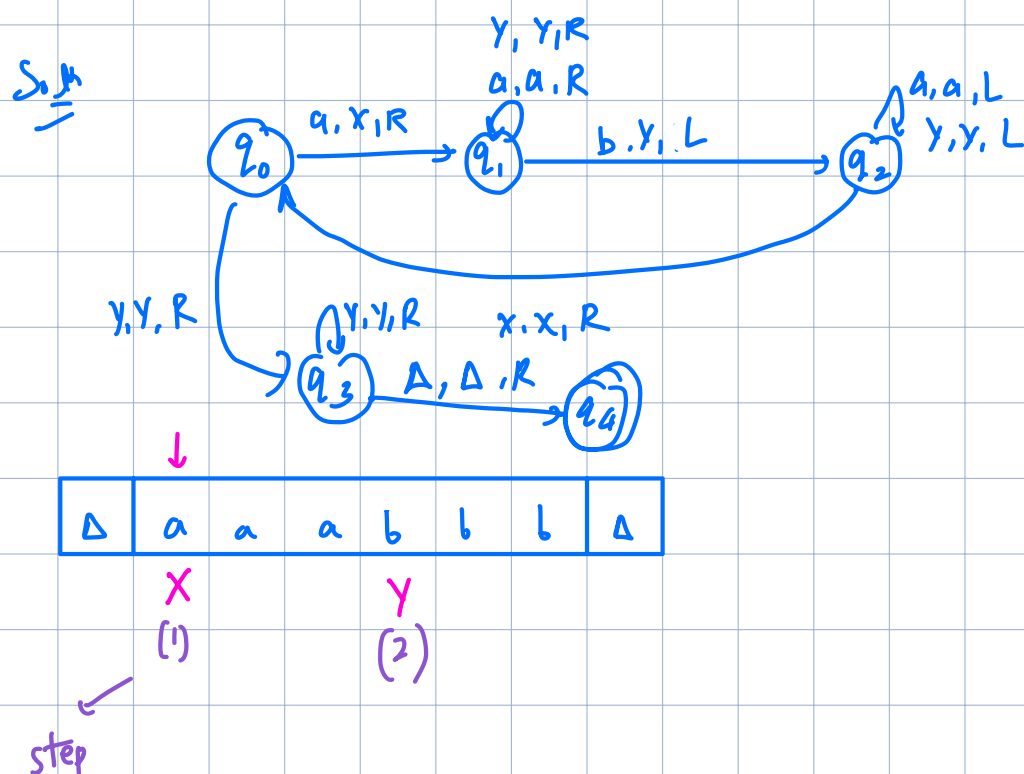
$\Delta \in \Gamma \rightarrow$ represents blank

$$F \subseteq Q$$

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, L, S\}$$

- the input tape is unbounded hence any number of Left and Right moves are possible.
- the transitions are deterministic. ie atleast 1 move for each config is defined.
- there is no special input or output files.

Q1 Design and construct TM for $L = \{w \mid a^n b^n, n \geq 1\}$



★ PDA limitation: can't recognize langs of form $a^n b^n c^n$

⇒ Context-Sensitive Language: (TYPE-I) (length increasing / non-contracting)

A context-sensitive grammar (CSG) is an unrestricted grammar in which no production is length-decreasing. In other words, every production is of the form $\alpha \rightarrow \beta$ where $|\beta| \geq |\alpha|$

$$\begin{aligned} \alpha &\in (\Sigma \cup V_n)^* V_n (\Sigma \cup V_n)^* \\ \beta &\in (\Sigma \cup V_n)^+ \end{aligned}$$

↓
non-terminal ↓
terminal

A language is a context-sensitive language (CSL) if it can be generated by a CSG.

Note:

$$\begin{aligned} \alpha A \beta &\rightarrow \alpha \delta \beta \\ \alpha, \beta &\in (\Sigma \cup V_n)^* \\ A &\in V_n \\ \delta &\in (\Sigma \cup V_n)^+ \end{aligned}$$

- In terms of grammar $G = (V, T, S, P)$

$$\alpha \rightarrow \beta$$

$$\alpha, \beta \in (V \cup T)^+$$

⇒ Linear-Bounded Automata:

A linear bounded automata (LBA) is a 5 tuple $M = (Q, \Sigma, \Gamma, q_0, \delta)$ that is identical to a non-deterministic TM, with the following exception:

- There are 2 extra tape symbols $[$ and $]$, assumed not to be elements of the tape alphabet Γ .
- The initial configuration of M corresponding to input x is $q_0 [x]$ with the symbol $[$ in the leftmost square and the symbol $]$ is the first square of these brackets or to move its tape head to the left of the $[$ or to the right of the $]$

★ $PDA \leq LBA \leq TM$

or 7 tuple definition:

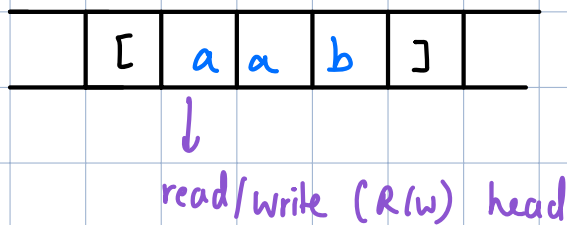
$(Q, \Sigma, \Gamma, \delta, q_0, \tau, M)$

↓ ↘ ↪ markers

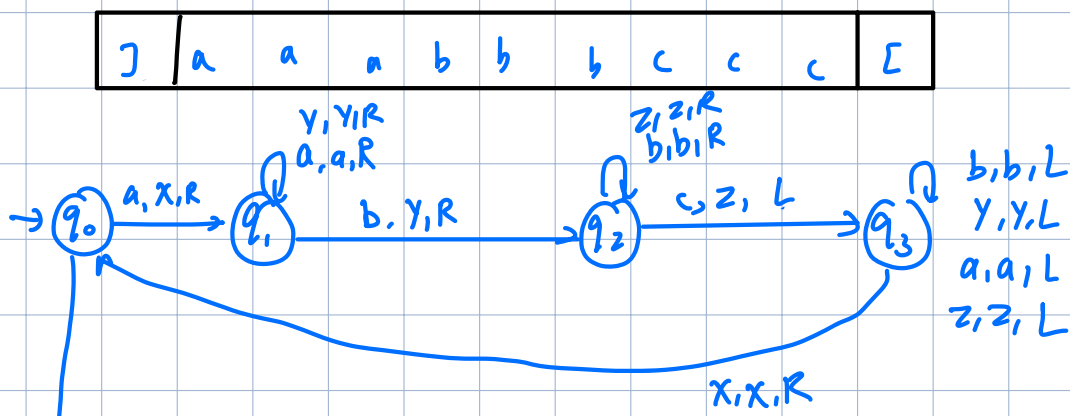
input alphabet tape alphabet (left end marker)
(right end marker)

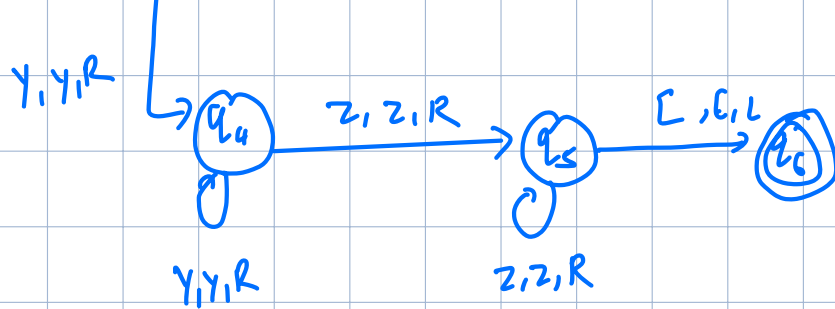
$$\delta : \mathcal{G} \times \Gamma \rightarrow (\mathcal{G} \times \Gamma) \times \{L, R\}$$

→ Model of LBA:



Q2. LBA for $a^n b^n c^n \mid n \geq 1$





Note: • Recursively enumerable languages are those that can be accepted by TM
• Recursive languages are those that can be decided by TM.

8.1 | RECURSIVELY ENUMERABLE AND RECURSIVE

Definition 8.1 Accepting a Language and Deciding a Language

A Turing machine T with input alphabet Σ accepts a language $L \subseteq \Sigma^*$ if $L(T) = L$. T decides L if T computes the characteristic function $\chi_L : \Sigma^* \rightarrow \{0, 1\}$. A language L is *recursively enumerable* if there is a TM that accepts L , and L is *recursive* if there is a TM that decides L .

Recursively enumerable languages are sometimes referred to as Turing-acceptable, and recursive languages are sometimes called Turing-decidable, or simply decidable.

As the discussion in Section 7.3 suggested, trying to accept L and trying to decide L are two ways of approaching the membership problem for L :

Given a string $x \in \Sigma^*$, is $x \in L$?

To decide L is to solve the problem conclusively, because for every $x \in \Sigma^*$, deciding whether $x \in L$ is exactly what must be done in order to determine the value of $\chi_L(x)$. Accepting L may be less conclusive. For a string $x \notin L$, an algorithm accepting L works correctly as long as it doesn't report that $x \in L$; but not reporting that $x \in L$ is less informative than reporting that $x \notin L$. At this stage, however, though the definitions are clearly different, it is not obvious whether there are really languages satisfying one and not the other.

We will be able to resolve these questions before this chapter is over. In this first section, we establish several facts that will help us understand the relationships between the two types of languages, starting in Theorem 8.2 with the fundamental relationship that we figured out in Section 7.3.

In proving the statements in this section, as well as in later parts of the chapter, we will take advantage of the Church-Turing thesis. For example, to show that a language L is recursively enumerable, it will be sufficient to describe an algorithm (without describing exactly how a TM might execute the algorithm) that answers yes if the input string is in L and either answers no or doesn't answer if the input string is not in L .

Theorem 8.2

Every recursive language is recursively enumerable.

Proof

Suppose T is a TM that decides $L \subseteq \Sigma^*$. An algorithm to accept L is the following: Given $x \in \Sigma^*$, execute T on input x . T will halt and produce an output; if the output is 1 (i.e., if $\chi_L(x) = 1$), accept, and if the output is 0, reject.

The reason the converse of Theorem 8.1 is not obviously true, and will turn out to be false, is that if T accepts L , there may be input strings not in L that cause T to loop forever. The closest we can come to the converse statement is Theorem 8.3.

Theorem 8.3

If $L \subseteq \Sigma^*$ is accepted by a TM T that halts on every input string, then L is recursive.

Proof

If T halts on every input string, then the following is an algorithm for deciding L : Given $x \in \Sigma^*$, if T accepts x , return 1, and if T rejects x , return 0.

A consequence of Theorem 8.3 is that if L is accepted by a nondeterministic TM T , and if there is no input string on which T can possibly loop forever, then L is recursive. The reason is that we can convert T to a deterministic TM T_1 as described in Section 7.7. The construction guarantees that if there are no inputs allowing T to loop forever, then T_1 halts on every input.

Theorem 8.4

If L_1 and L_2 are both recursively enumerable languages over Σ , then $L_1 \cup L_2$ and $L_1 \cap L_2$ are also recursively enumerable.

Proof

Suppose that T_1 is a TM that accepts L_1 , and T_2 is a TM that accepts L_2 . The algorithms we describe to accept $L_1 \cup L_2$ and $L_1 \cap L_2$ will both process an input string x by running T_1 and T_2 simultaneously on the string x . We might implement the algorithm by building a two-tape TM that literally simulates both TMs simultaneously, one on each tape, or we might simply use transition diagrams for both TMs to trace one move of each on the input string x , then two moves of each, and so forth.

To accept $L_1 \cup L_2$, the algorithm is simply to wait until one of the two TMs accepts x , and to accept only when that happens. If one rejects, we abandon it and continue with the other. If both eventually reject, we can reject and halt, though this is not required in order to accept $L_1 \cup L_2$. If both TMs loop forever, then we never receive an answer, but the only case in which this can occur is when $x \notin L_1 \cup L_2$.

To accept $L_1 \cap L_2$, the algorithm is to wait until both TMs accept x , and to accept only when that happens. This time, if one TM accepts, we continue with the other; if either TM ever rejects, we can reject and halt, though again this is not necessary.

Theorem 8.5

If L_1 and L_2 are both recursive languages over Σ , then $L_1 \cup L_2$ and $L_1 \cap L_2$ are also recursive.

Proof

See Exercise 8.1.

Theorem 8.6

If L is a recursive language over Σ , then its complement L' is also recursive.

Proof

If T is a TM that computes the characteristic function χ_L , the TM obtained from T by interchanging the two outputs computes the characteristic function of L' .

Theorem 8.7

If L is a recursively enumerable language, and its complement L' is also recursively enumerable, then L is recursive (and therefore, by Theorem 8.6, L' is recursive).

Proof

If T is a TM accepting L , and T_1 is another TM accepting L' , then here is an algorithm to decide L : For a string x , execute T and T_1 simultaneously on input x , until one halts. (One will eventually accept, because either $x \in L$ or $x \in L'$.) If the one that halts first is T , and it accepts, or the one that halts first is T_1 , and it rejects, return 1; otherwise return 0.

Theorem 8.7 implies that if there are any nonrecursive languages, then there must be languages that are not recursively enumerable. (Otherwise, for every language L , both L and L' would be recursively enumerable and therefore recursive.)

Suppose we have a TM T that accepts a language L . The possibility that T might loop forever on an input string x not in L might prevent us from using T to decide L . (This is why the converse of Theorem 8.2 is not obviously true, as we observed above.) The same possibility might also prevent us from using T in an algorithm to accept L' (this is why Theorem 8.6 is stated for recursive languages, not recursively enumerable languages), because accepting L' requires answering yes for strings in L' , and these are the strings for which T might not return an answer.

For the language L accepted by T , according to Theorems 8.6 and 8.7, the two problems are equivalent: If we could find another TM to accept L' , then we could find another TM to decide L , and conversely. We will see later in this chapter that the potential difficulty cannot always be eliminated. There *are* languages that can be accepted by TMs but not decided. These are the same languages that can be accepted by TMs but whose complements cannot; they are languages that can be accepted, but only by TMs that loop forever for some inputs not in the language.

Definition 8.8 A TM Enumerating a Language

Let T be a k -tape Turing machine for some $k \geq 1$, and let $L \subseteq \Sigma^*$. We say T enumerates L if it operates such that the following conditions are satisfied.

1. The tape head on the first tape never moves to the left, and no nonblank symbol printed on tape 1 is subsequently modified or erased.
2. For every $x \in L$, there is some point during the operation of T when tape 1 has contents

$$x_1 \# x_2 \# \dots \# x_n \# x \#$$

for some $n \geq 0$, where the strings $x_1, x_2, \dots, x_n$ are also elements of L and $x_1, x_2, \dots, x_n, x$ are all distinct. If L is finite, then nothing is printed after the $\#$ following the last element of L .

This idea leads to a characterization of recursively enumerable languages, and with an appropriate modification involving the order in which the strings are listed, it can also be used to characterize recursive languages.

$\Rightarrow$ Unrestricted Grammar:

An unrestricted grammar is a 4-tuple $G = (V, \Sigma, S, P)$, where V and Σ are disjoint sets of variables and terminals, respectively. S is an element of V → start symbol, and P is a set of productions of the form

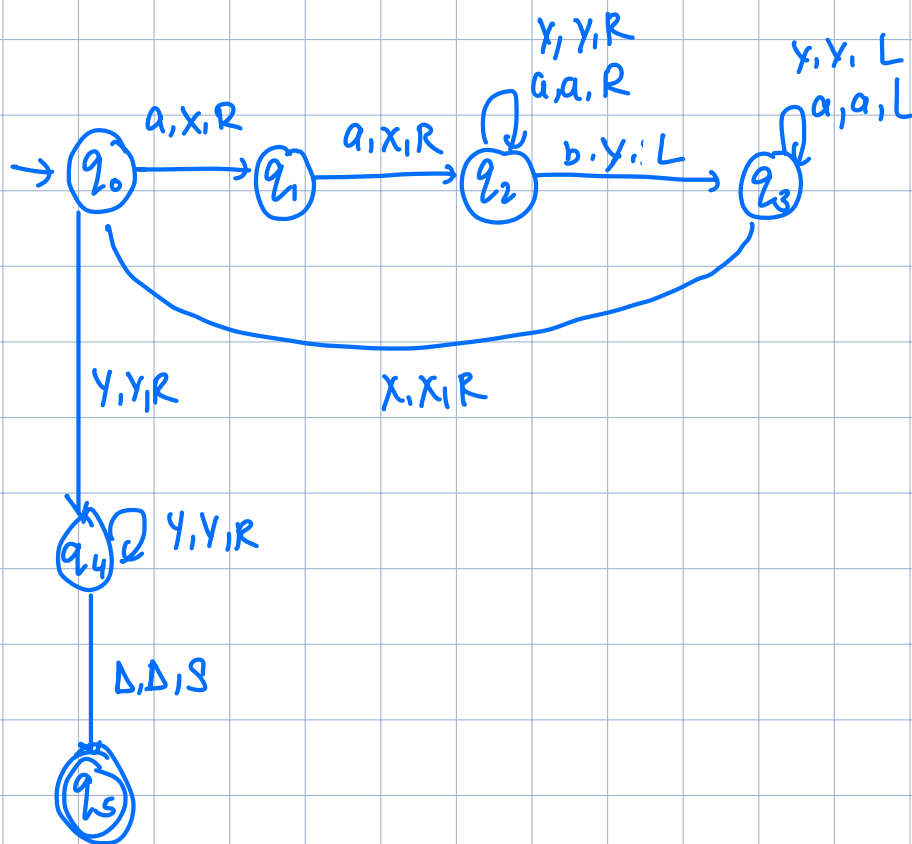
$$\boxed{\alpha \rightarrow \beta}$$

where, $\alpha, \beta \in (V \cup \Sigma)^*$ and α consists at least one variable.

Q.3. TM for $a^{2n} b^n$.

Solⁿ

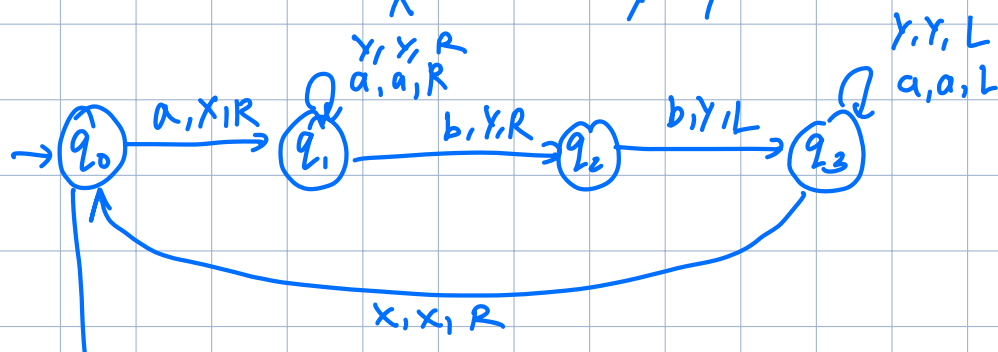
Δ Δ a a a a b b Δ Δ
 x x y
 x x y

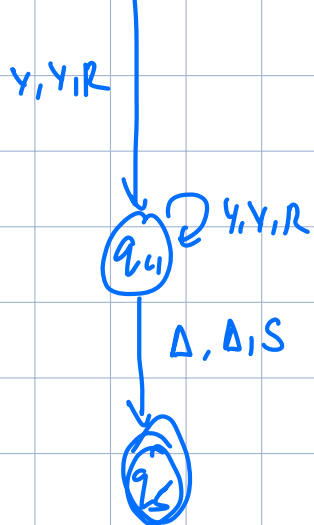


Q4. TM for $a^n b^{2n}$

Solⁿ

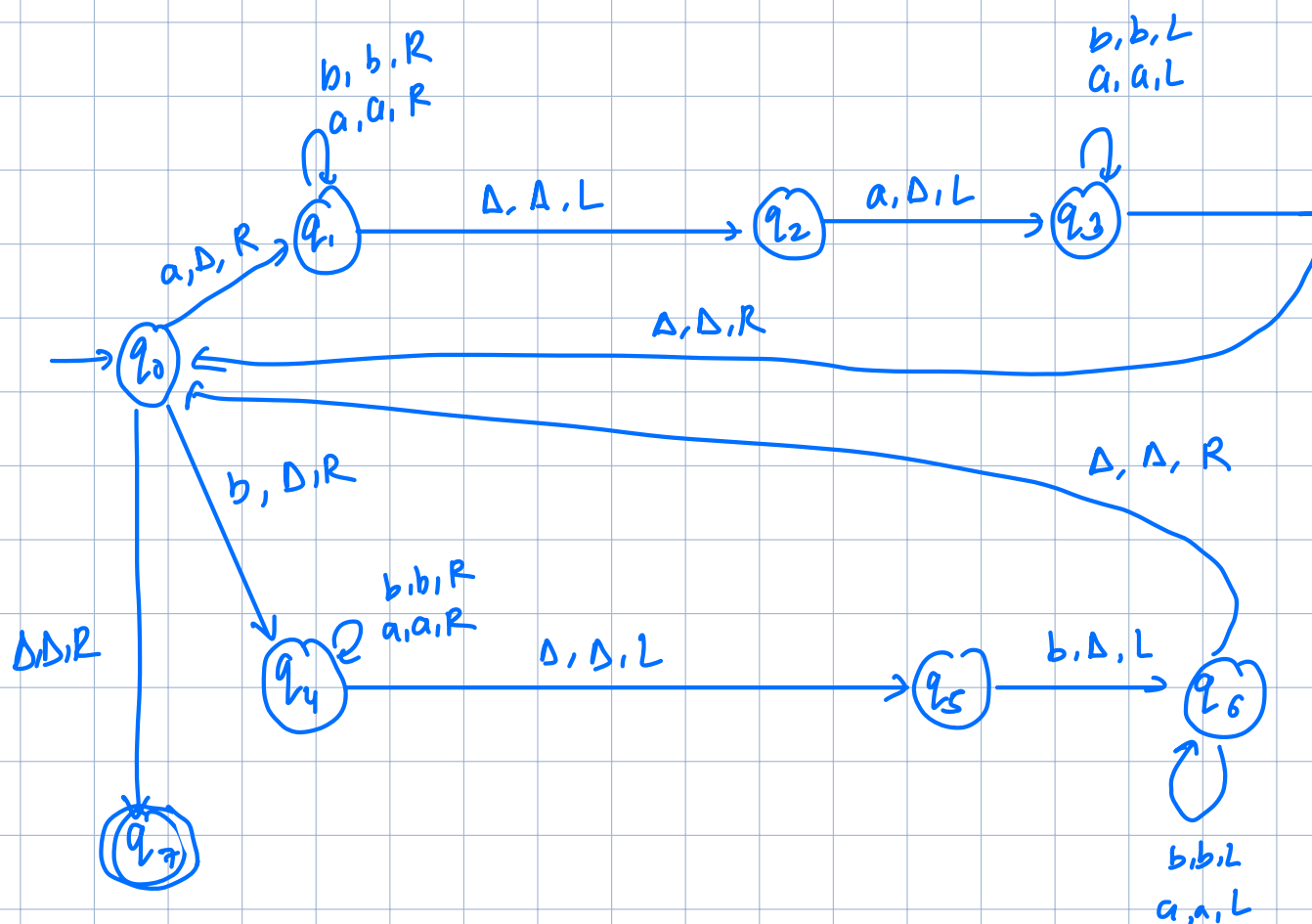
Δ Δ a a b b b b Δ Δ
 x y y
 x y y





Q5. TM for even Palindrome

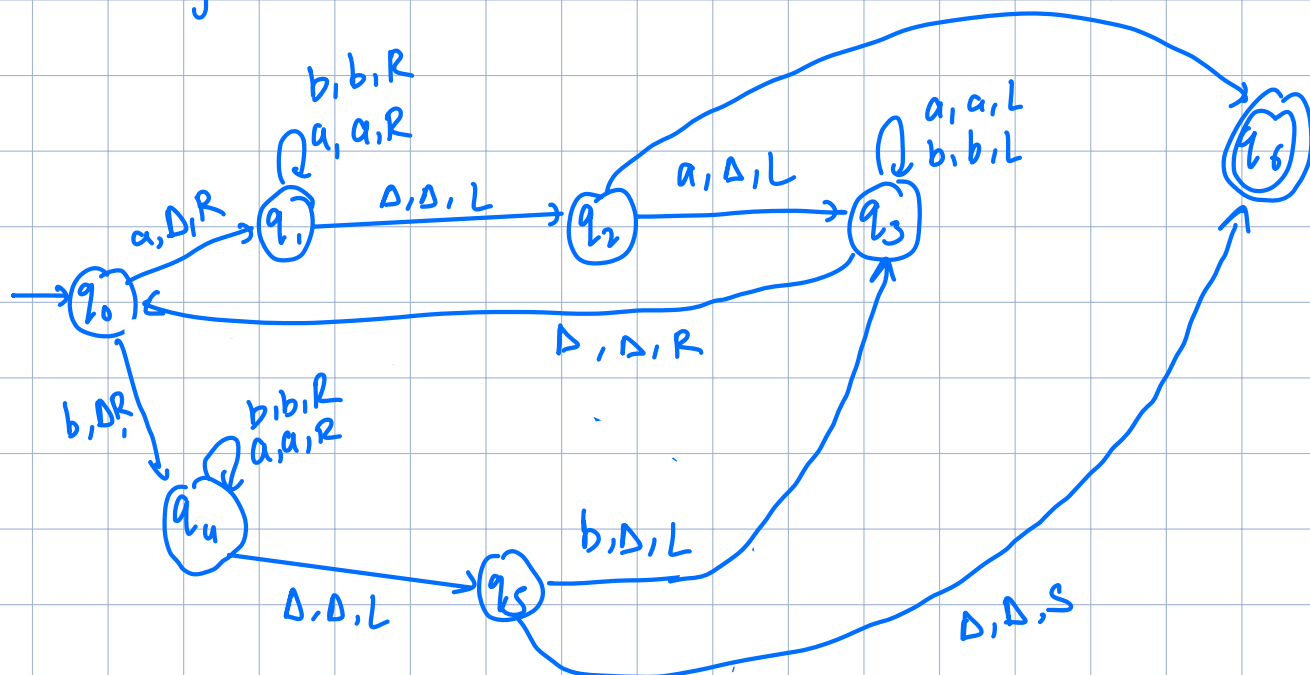
Soln eg: $\Delta \Delta a b a a b a \Delta \Delta$
 Δ



Q6. TM for odd palindrom (wawr, wbwr)

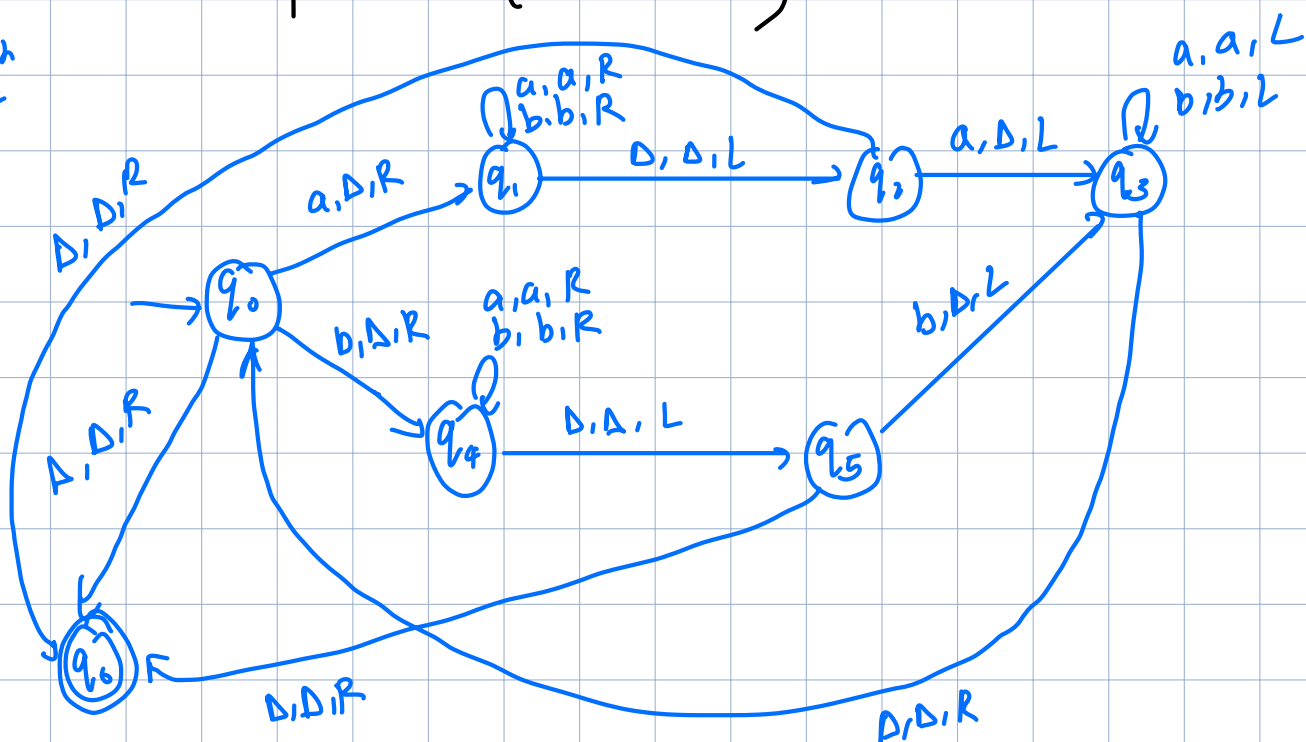
Soln

eg: $\Delta \Delta a b a b a \Delta \Delta \Delta, \Delta, S$



Q7. TM for all palindromes (odd and even)

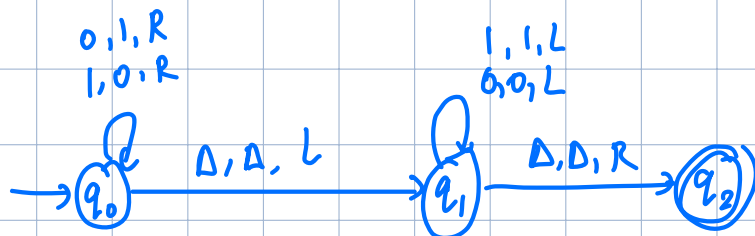
Soln



Q8. TM for 1's and 2's complement

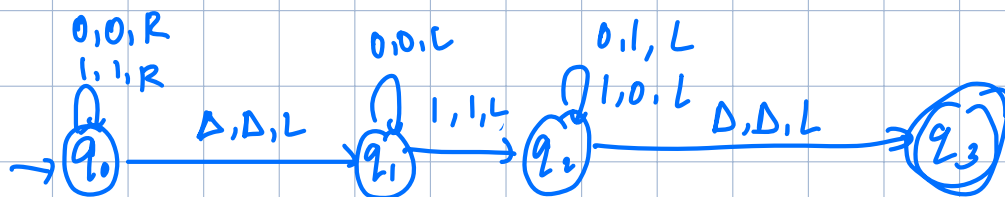
Solⁿ

1's complement = $\begin{array}{cccc} 1 & 0 & 1 & 0 \\ \uparrow & \uparrow & \uparrow & \uparrow \\ 0 & 1 & 0 & 1 \end{array}$



2's complement

$\begin{array}{cccccc} 1 & 0 & 0 & 1 & 0 & 0 \\ & & & 0 & 1 & 1 \\ + & & & & & 1 \\ \hline 0 & 1 & 1 & 1 & 0 & 0 \end{array}$



Q9. TM for $L = \{w \mid w \in (a+b)^A\}$

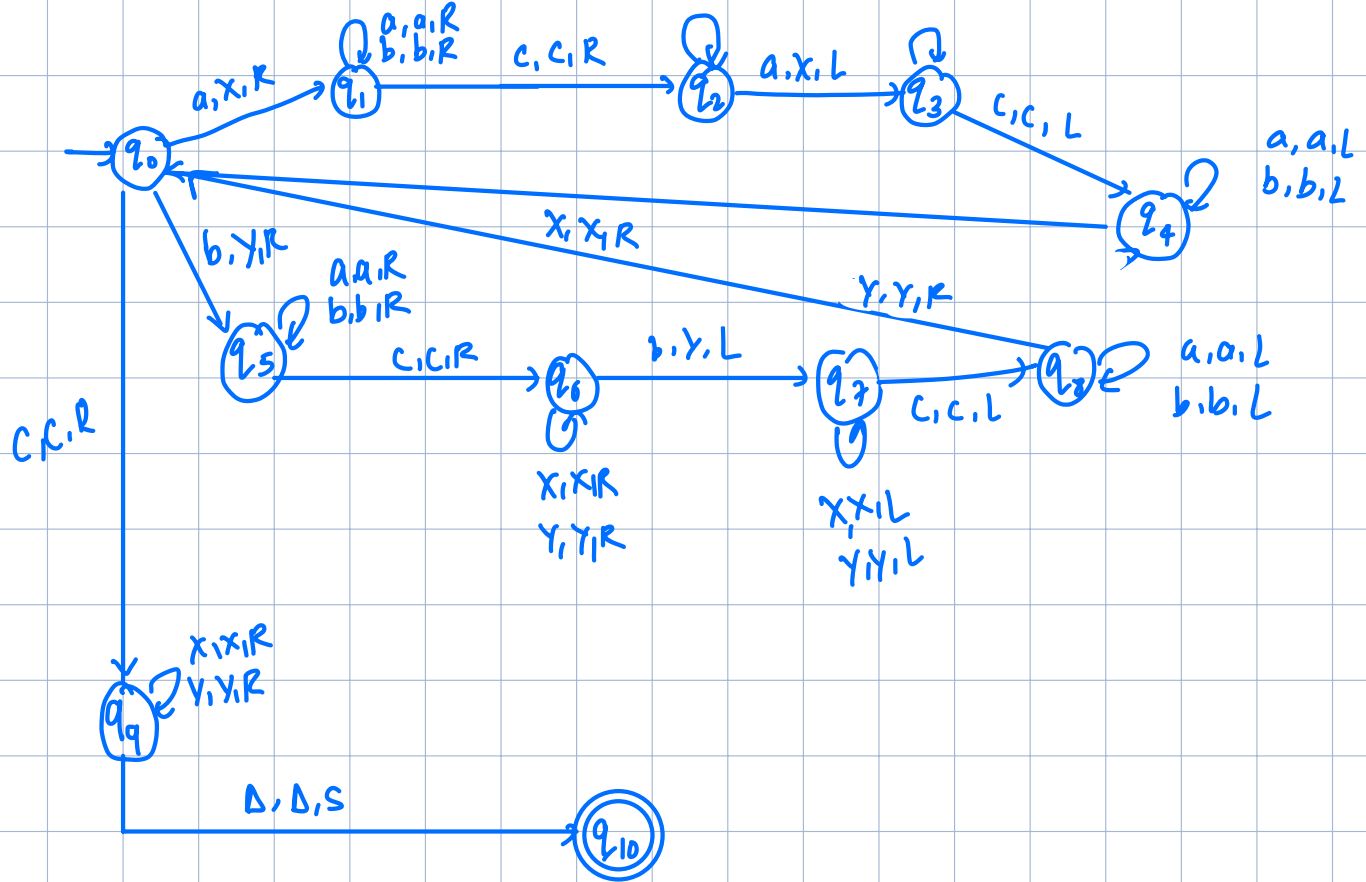
Solⁿ

let $w = abab$

$abab \in L$

y, y, R
 x, x, R

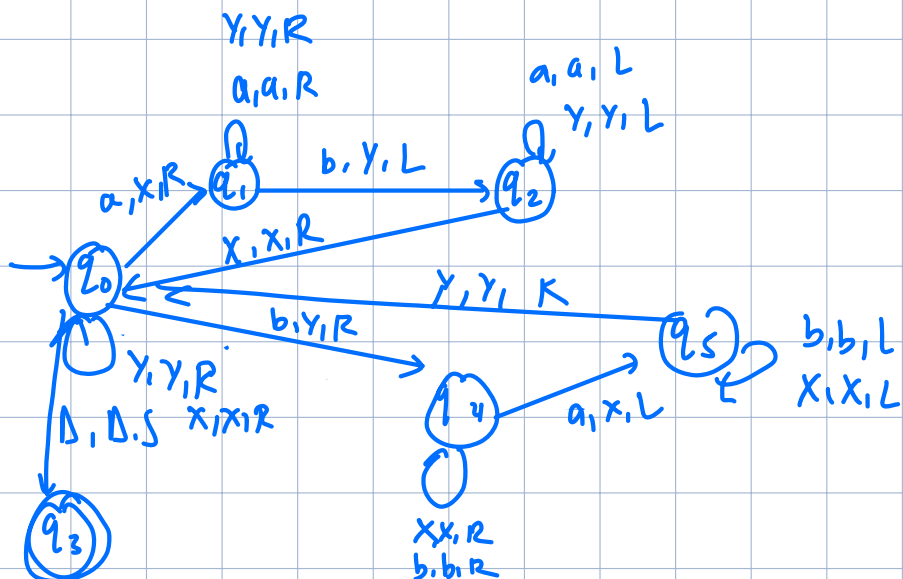
x, x, L
 y, y, L



Q10. TM for equal no. of a's and b's

Solⁿ $L = \{ abab, aabb, aaabbb, \dots \}$

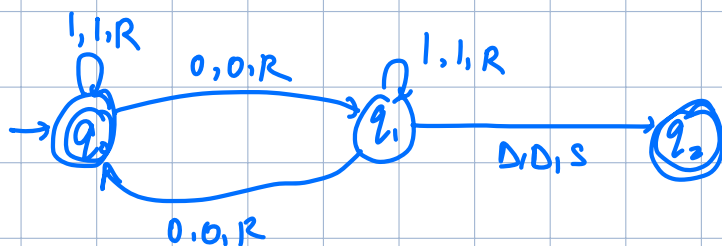
Δ	a	a	b	b	Δ
	x		y		
		x		y	



Q11. TM for accepting even no. of 0's and any no. of 1's.

Solⁿ

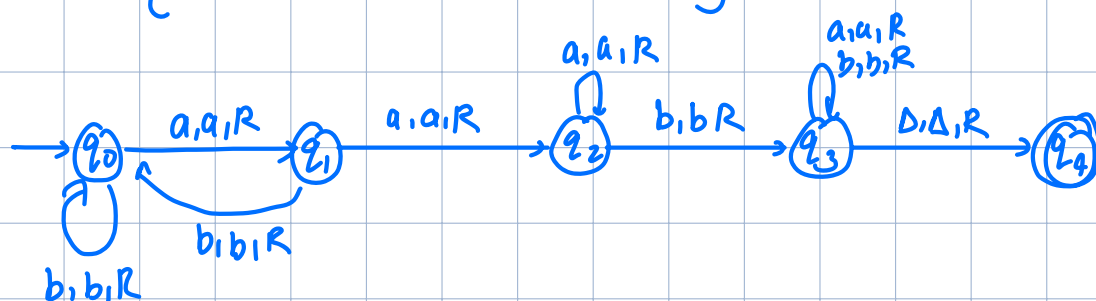
$$L = \{\epsilon, 0011, 000011, \dots\}$$



Q12. TM which contains substring aab

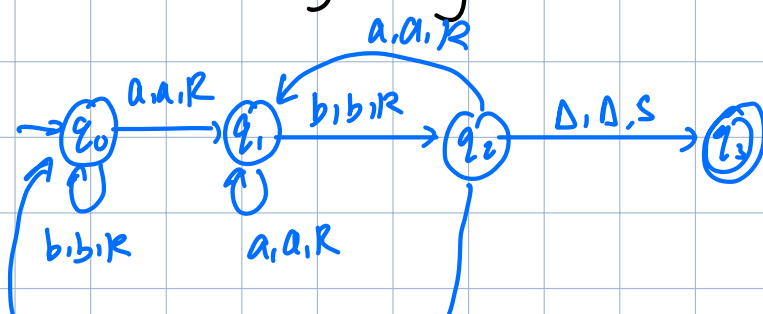
Solⁿ

$$L = \{aab, baab, aaab, \dots\}$$



Q13. Design TM for string ending in 'ab'.

Solⁿ

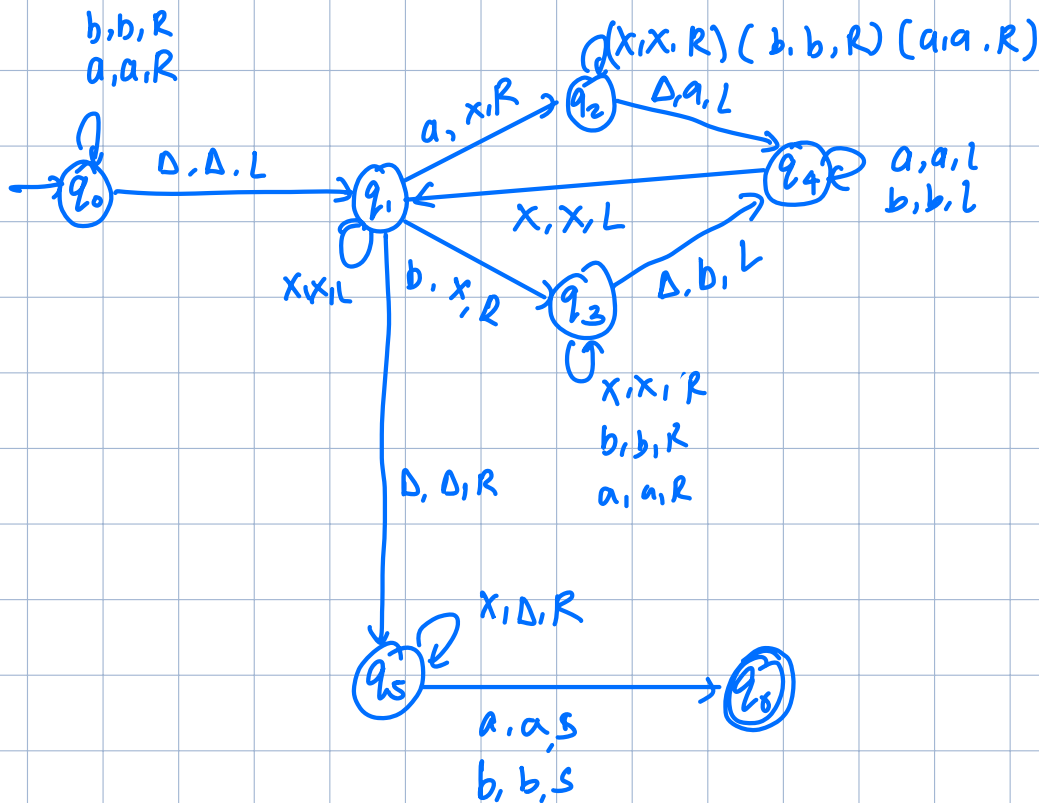


b, b, R

Q14. TM for reversing a string

Soln

Δ Δ a b a b Δ Δ Δ Δ
x < < < < b a b a



⇒ Variations of Turing Machines:

- (1) 2 way infinite tape TM
- (2) Multitape TM —
- (3) Non-deterministic TM —

(4) Multidimensional TM ✓

(5) Multihead TM ✓

(6) offline TM ✓

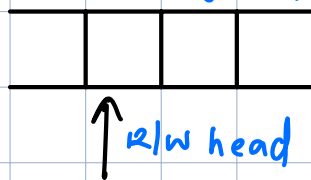
(7) Multi track TM ✓

(8) Jumping TM

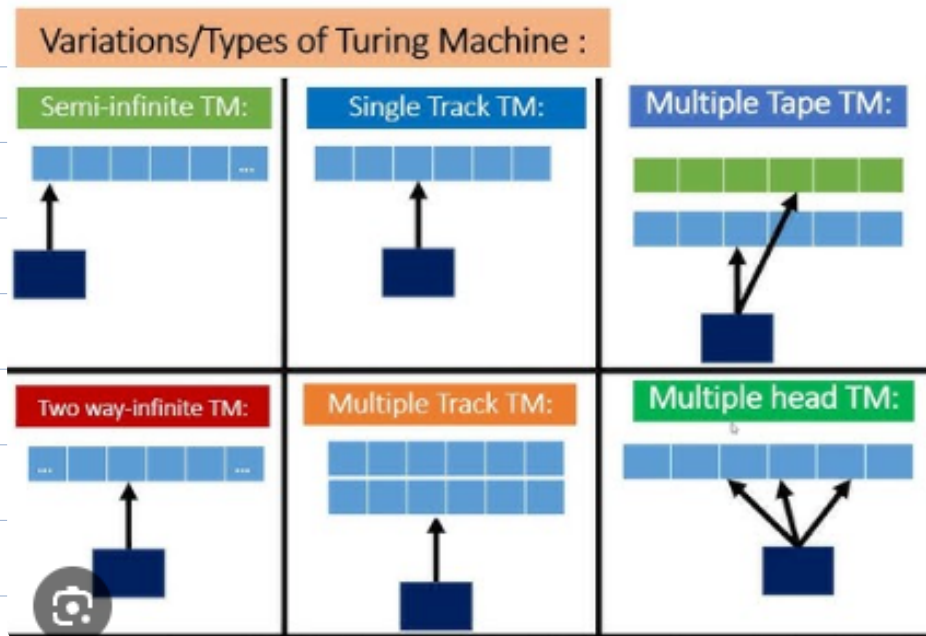
(9) TM with stay option , (10) Non-halting TM

(11) Universal TM

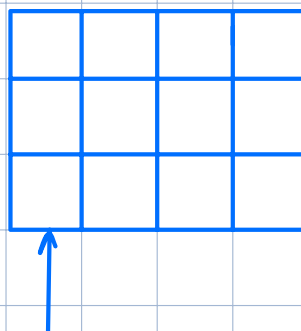
(12) Multistack TM



Basic structure of TM

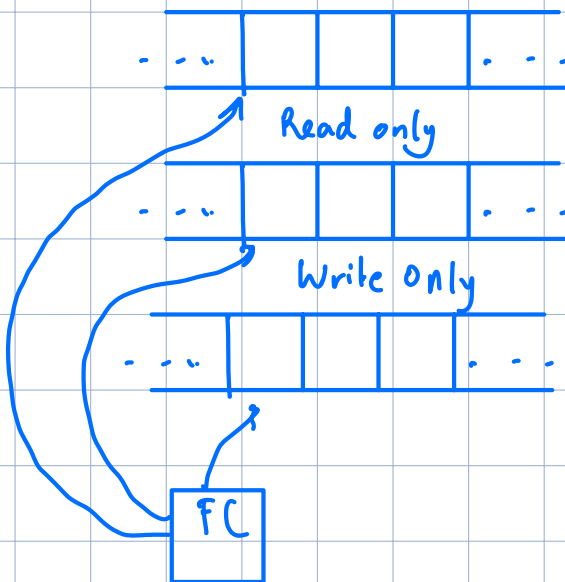


Multi-dimensional

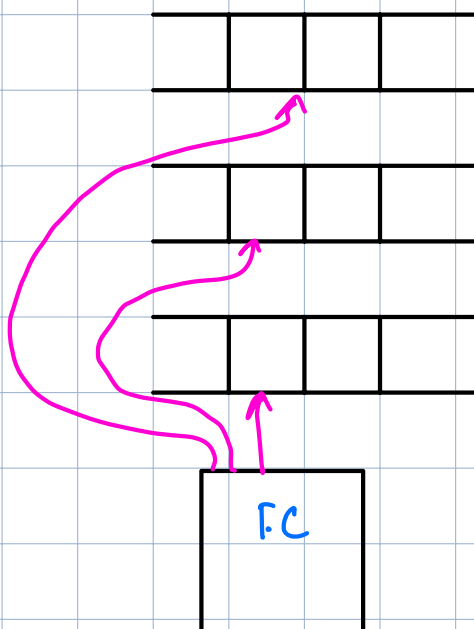




Offline TM



(2) Multitape TM:



- consists of k -tapes, k -heads

$$M = \{Q, \Sigma, \Gamma, \delta, q_0, \Delta, F\}$$

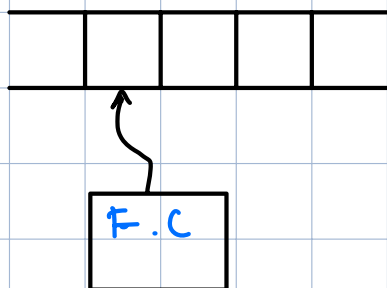
$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma \times (\{R, L, S\})^k$$

- Applications:

(1) Efficient algorithm simulation

(2) Compiler design

(2) Non-Deterministic TM:



if $T = (Q, \Sigma, \Gamma, q_0, \delta)$

and $(q, a) \in Q \times (\Gamma \cup \{\Delta\})$

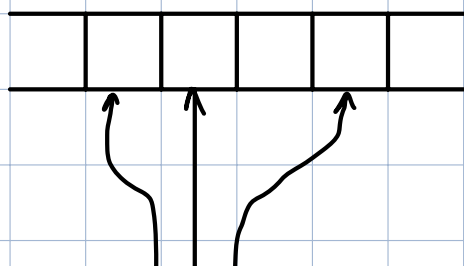
then $\delta(q, a)$ is a finite subset, not an element, of $(Q \cup \{h, r\}) \times (\Gamma \cup \{\Delta\}) \times \{L, S\}$

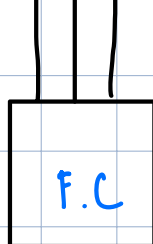
- Applications :

(1) Complexity theory. AI , "guess and verify" problems

(2) Cryptography

(3) Multi-head TM:





$$T = (Q, \Sigma, \Gamma, q_0, \delta)$$

$p_i, p_j \dots \rightarrow$ pointers from each head.

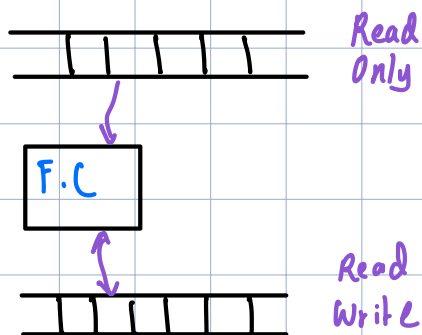
$$\delta: Q \times ((\Gamma)_{p_1}, (\Gamma)_{p_2} \dots (\Gamma)_{p_n}) \rightarrow Q \times ((\Gamma_{p_1} \times \{L, R, S\}), (\Gamma_{p_2} \times \{L, R, S\}), \dots, (\Gamma_{p_n} \times \{L, R, S\}))$$

Applications:

- (1) Parallel string processing
- (2) Lexical analysis
- (3) Memory scanning algorithms

(4) Offline TM

- multi-tape variant
- It has 2 tapes:
 - (1) Read-only input file tape
 - (2) Read-write tape.



- Application:

- (1) To prove language properties.
- (2) Analyse space time complexity of algorithms.

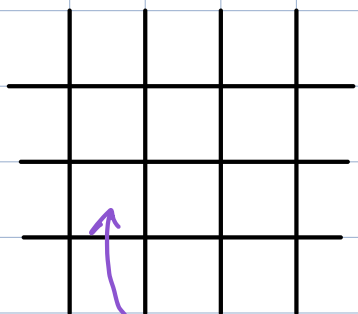
(5) Multidimensional TM:

- extends classical 1D TM to d -dimensional tapes ($d \geq 2$), modelling computation with multi-dimensional memory access

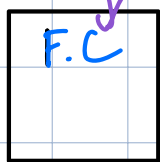
- the tape is $\mathbb{Z}^d$ grid insted of 1D line

let $M = (Q, \Sigma, \delta, \overset{\text{extended alphabet } \subseteq \Gamma}{\Gamma}, q_0, D, F)$

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, U, D\}$$



↪ 2D address schema.



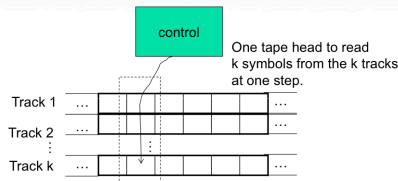
Applications:

- (1) Image processing (2D i/p)
- (2) Model 2D cellular automata

(6) Multi-track TM:

- TM with multiple tracks, but just one unified tape head
- one tape head to read k symbols from k tracks at one step

- TM with multiple tracks, but just one unified tape head

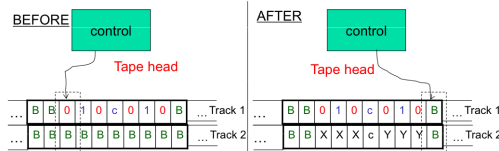


18

Multi-Track TMs

- TM with multiple "tracks" but just one head

E.g., TM for $\{w^c w \mid w \in \{0,1\}^*\}$
but w/o modifying original input string



Second track mainly used as a scratch space for marking

19

Multi-track TMs are equivalent to basic (single-track) TMs

- Let M be a single-track TM
 - $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$
- Let M' be a multi-track TM (k tracks)
 - $M' = (Q', \Sigma', \Gamma', \delta', q'_0, B, F')$
 - $\delta'(q_i, \langle a_1, a_2, \dots, a_k \rangle) = (q_j, \langle b_1, b_2, \dots, b_k \rangle, L/R)$
- Claims:
 - For every M , there is an M' s.t. $L(M) = L(M')$.
 - (proof trivial here)

Applications:

- (i) Symbol grouping
- (ii) Binary arithmetic
- (iii) Data encoding

(7) Jumping TM:

- tape head can jump to specific locations determined by jumping factor.

Jumping TM

$\{a|b|b|b|\} \Rightarrow \{a|b|b|b|\}$ (a changes to b, b remains b itself)

FCU (set of all states Q)

Here $\exists q_0 \in Q$
 $\forall Q \in FCU$

FCU (set of all states Q)

Here $\exists q_1 \in Q$
 $\forall Q \in FCU$

$\delta(q_0, a) = (q_1, b, R, 3)$

$\delta: Q \times \Gamma^* \rightarrow Q \times \Gamma \times \{L, R, J\} \times \{n\}$
no of jumps

$M = (Q, \Sigma, q_0, \delta, F, B, \Gamma)$

Q = set of all finite set of state
 Σ = set of symbols
 $q_0 \Rightarrow$ initial state, i.e. $q_0 \in Q$
 $\delta \Rightarrow$ transition function
 $B \Rightarrow$ Blank character
 $F \Rightarrow$ set of all final states

Application

- 1) Computer architecture $\Rightarrow$ Modelling RAM
- 2) Complexity theory $\Rightarrow$ Algorithm benchmarking
- 3) Biology $\Rightarrow$ DNA splicing
- 4) Security $\Rightarrow$ Sparse data attack

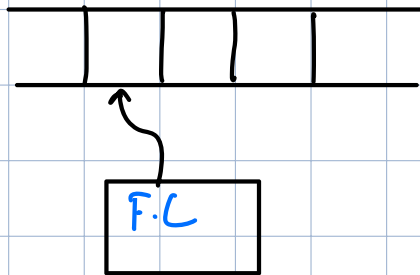
(8) TM with stay option:

— in a standard TM, the tape can:

- move left (L)
- move right (R)

— in a TM with stay option, the head can also:

- stay (S) on the same tape cell after reading/writing



$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$$

Applications:

- (1) Simpler algorithm design
- (2) State-based computation
- (3) Formal proofs

(9) Non-halting TM:

↳ enters ∞ loops for certain inputs.

Non-halting Turing machine:

↳ enters ∞ loops for certain inputs

Mathematical definition

Standard TM

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

w → machine → non-halting if sequences of moves is ∞

$$ID_0 \vdash ID_1 \vdash ID_2 \vdash ID_3 \dots$$

initial config

$q_0 w$

For every $n \geq 0$, there exists a valid transition δ such that ID_n yields ID_{n+1}

no ID_n contains a halting state

no further moves

Q

final state

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

↳ must be a total function ($q \rightarrow q' \rightarrow q''$)

Simulation using standard TM.

Feed UTM two things

description of NHTM ↗
input w ↖

→ now UTM mimics M

→ if M goes right, UTM updates its tape to go right

...

⋮

Applications:

(1) OS

(2) Reactive systems

(3) Modelling undecidable problems

(10) Universal TM:

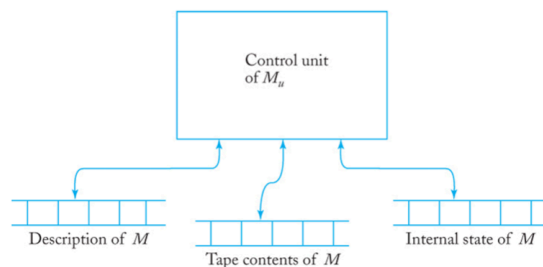
Universal turing machine:

A *universal* Turing machine is a Turing machine T_u that works as follows. It is assumed to receive an input string of the form $e(T)e(z)$, where T is an arbitrary TM, z is a string over the input alphabet of T , and e is an encoding function whose values are strings in $\{0, 1\}^*$. The computation performed by T_u on this input string satisfies these two properties:

1. T_u accepts the string $e(T)e(z)$ if and only if T accepts z .
2. If T accepts z and produces output y , then T_u produces output $e(y)$.

Applications of Universal Turing Machine:

1. General-purpose computers
2. Program execution and interpretation
3. Software portability
4. Complexity analysis
5. Computability theory
6. Operating systems
7. Emulation and virtualization
8. Programming language design
9. Artificial intelligence research
10. Algorithm analysis



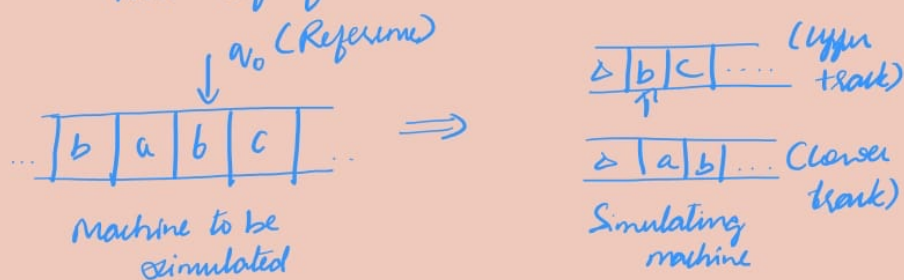
M_u = universal tm

M = turing machine given as input along with string

① Semi-infinite TM:

A Semi-Infinite Turing machine is a variant of Turing machine that is unbounded on one side, generally the right side.

- 1) The tape is infinite only in one direction and has a boundary in the other direction
- 2) The leftmost cell is fixed, one head cannot move left from this cell.



$$M = (Q, \Sigma, \Gamma, \delta, q_0)$$

Q = finite set of states

Σ = input alphabets without blank

Γ = tape alphabets

q_0 = Initial state

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

If head points to the left most cell q_0 left transition is not allowed.

Applications:

- (1) Simplified algorithm design
- (2) Compiler and interpreter Design
- (3) Language recognition

(12) Multi-stack TM:

- a multistack TM is a variant of a TM that replaces the tape with k -stacks (where $k \geq 2$), and computation is performed using stack operation.

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

$$\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma^k \rightarrow Q \times (\Gamma^*)^k$$

4. How Multi-Stack TM Works

Two-Stack Simulation of a Tape

- Stack 1: Symbols to the left of the head
- Stack 2: Current symbol + symbols to the right

Head Movement

- Move Right $\rightarrow$ pop from Stack 2, push to Stack 1
- Move Left $\rightarrow$ pop from Stack 1, push to Stack 2

This perfectly simulates an infinite tape.

5. Computational Power

Model	Power
1 Stack	Equivalent to PDA
2 Stacks	Equivalent to TM
$k \geq 2$ Stacks	Equivalent to TM

🚫 Multiple stacks do not increase power, only simplify implementation.

Applications:

- (1) Programming language implementation
- (2) Expression evaluation
- (3) Compiler design
- (4) Memory management

EXAMPLE 8.12 A Grammar Generating $\{a^n b^n c^n \mid n \geq 1\}$

The previous example used the idea of a variable moving through the string and operating on it. In this example we use a similar left-to-right movement, although there is no explicit "operator" like the variable D , as well as another kind arising from the variables rearranging themselves. Like the previous example, this one uses the variable L to denote the left end of the string.

We begin with the productions

$$S \rightarrow SABC \mid LABC$$

which allow us to obtain strings of the form $L(ABC)^n$, where $n \geq 1$. Next, productions that allow the variables A , B , and C to arrange themselves in alphabetical order:

$$BA \rightarrow AB \quad CB \rightarrow BC \quad CA \rightarrow AC$$

Finally, productions that allow the variables to be replaced by the corresponding terminals, provided they are in alphabetical order:

$$LA \rightarrow a \quad aA \rightarrow aa \quad aB \rightarrow ab \quad bB \rightarrow bb \quad bC \rightarrow bc \quad cC \rightarrow cc$$

Although nothing forces the variables to arrange themselves in alphabetical order, doing so is the only way they can ultimately be replaced by terminals.

None of the productions can rearrange existing terminal symbols. This means, for example, that if an a in the string, which must come from an A , has a terminal right before

EXAMPLE 8.11 A Grammar Generating $\{a^{2^k} \mid k \in \mathbb{N}\}$

Let $L = \{a^{2^k} \mid k \in \mathbb{N}\}$. L can be defined recursively by saying that $a \in L$ and that for every $n \geq 1$, if $a^n \in L$, then $a^{2n} \in L$. Using this idea to obtain a grammar means finding a way to double the number of a 's in the string obtained so far. The idea is to use a variable D that will act as a "doubling operator." D replaces each a by two a 's, by means of the production $Da \rightarrow aaD$. At the beginning of each pass, D is introduced at the left end of the string, and we think of each application of the production as allowing D to move past an a , doubling it in the process. The complete grammar has the productions

$$S \rightarrow LaR \quad L \rightarrow LD \quad Da \rightarrow aaD \quad DR \rightarrow R \quad L \rightarrow \Lambda \quad R \rightarrow \Lambda$$

Beginning with the string LaR , the number of a 's will be doubled every time a copy of D is produced and moves through the string. Both variables L and R can disappear at any time. There is no danger of producing a string of a 's in which the action of one of the doubling operators is cut short, because if R disappears when D is present, there is no way for D to be eliminated. The string $aaaa$ has the derivation

$$\begin{aligned} S &\Rightarrow LaR \Rightarrow LDaR \Rightarrow LaaDR \Rightarrow LaaR \Rightarrow LDaaR \\ &\Rightarrow LaaDaR \Rightarrow LaaaaDR \Rightarrow LaaaaR \Rightarrow aaaaR \Rightarrow aaaa \end{aligned}$$

[THEOREM]: For every unrestricted grammar G , there is a Turing machine T with $L(T) = L(G)$

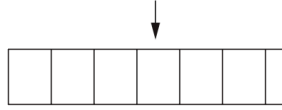
[THEOREM]: For every Turing machine T with input alphabet Σ , there is an unrestricted grammar G generating the language $L(T) \subseteq \Sigma^*$

[THEOREM]: Every context-sensitive language L is recursive

possible.

With these principles in mind, Turing isolated what he took to be the primitive steps that a human computer carries out during a computation:

1. Examining an individual symbol on the paper;
2. Erasing a symbol or replacing it by another;
3. Transferring attention from one symbol to another nearby symbol.



In our version of a Turing machine, which is similar though not identical to the one proposed by Turing, a single move is determined by the current state and the current tape symbol, and consists of three parts:

1. Changing from the current state to another, possibly different state;
2. Replacing the symbol in the current square by another, possibly different symbol;
3. Leaving the tape head on the current square, or moving it one square to the right, or moving it one square to the left if it is not already on the leftmost square.

Definition 7.1 Turing Machines

A Turing machine (TM) is a 5-tuple $T = (Q, \Sigma, \Gamma, q_0, \delta)$, where

Q is a finite set of states. The two *halt* states h_a and h_r are not elements of Q .

Σ , the input alphabet, and Γ , the tape alphabet, are both finite sets, with $\Sigma \subseteq \Gamma$. The *blank* symbol Δ is not an element of Γ .

q_0 , the initial state, is an element of Q .

δ is the transition function:

$$\delta : Q \times (\Gamma \cup \{\Delta\}) \rightarrow (Q \cup \{h_a, h_r\}) \times (\Gamma \cup \{\Delta\}) \times \{R, L, S\}$$

For a state $p \in Q$, a state $q \in Q \cup \{h_a, h_r\}$, two symbols $X, Y \in \Gamma \cup \{\Delta\}$, and a “direction” $D \in \{R, L, S\}$, we interpret the formula

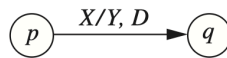
$$\delta(p, X) = (q, Y, D)$$

to mean that when T is in state p and the symbol on the current tape square is X , the TM replaces X by Y on that square, changes to state q , and either moves the tape head one square right, moves it one square left if the tape head is not already on square 0, or leaves it stationary, depending on whether D is R, L, or S, respectively. If the state q is either h_a or h_r , we say that this move causes T to *halt*. Once it halts, it cannot move, since δ is defined at (q, Y) only if $q \in Q$.

In this chapter we will return to drawing transition diagrams, similar to but more complicated than the diagrams for finite automata. The transition

$$\delta(p, X) = (q, Y, D)$$

will be represented by the diagram



⇒ Turing Machines as Language Acceptors:

If $T = (Q, \Sigma, \Gamma, q_0, \delta)$ is a TM and $x \in \Sigma^*$, x is accepted by T if

$$q_0 \Delta x \vdash^*_T w \# y$$

for some strings $w, y \in (\Gamma \cup \{\Delta\})^*$ (ie. if, starting in the initial configuration corresponding to input x , T eventually halts in the accepting state).

A language $L \subseteq \Sigma^*$ is accepted by T if $L = L(T)$, where
 $L(T) = \{x \in \Sigma^* \mid x \text{ is accepted by } T\}$

Saying that L is accepted by T means that for any $x \in \Sigma^*$, $x \in L$ if and only if x is accepted by T . This does not imply that if $x \notin L$, then x is rejected by T . The two possibilities for a string x not in $L(T)$ are that T rejects x and that T never halts, or loops forever, on input x .

A TM Accepting a Regular Language

EXAMPLE 7.3

Figure 7.3a shows the transition diagram of a finite automaton accepting $L = \{a, b\}^* \{ab\} \{a, b\}^* \cup \{a, b\}^* \{ba\}$, the language of all strings in $\{a, b\}^*$ that either contain the substring ab or end with ba .

Using a TM instead of an FA to accept this language might seem like overkill, but the transition diagram in Figure 7.4b illustrates the fact that every regular language can

230

CHAPTER 7 Turing Machines

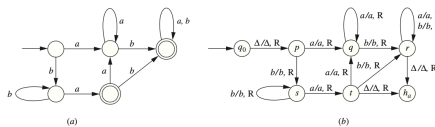


Figure 7.4
A finite automaton, and a Turing machine accepting the same language.

be accepted by a Turing machine, and that every FA can easily be converted into a TM accepting the same language.

Several comments about the TM are in order. First, it can process every input string in the way a finite automaton is forced to, moving the tape head to the right on each move and never modifying the symbols on the tape. This procedure will not work for languages that are not regular.

Second, the diagram in Figure 7.4b does not show moves to the reject state. There is one from each of the states p, q, r , and s (the nonhalting states that correspond to nonaccepting states in the FA), and they all look like this:



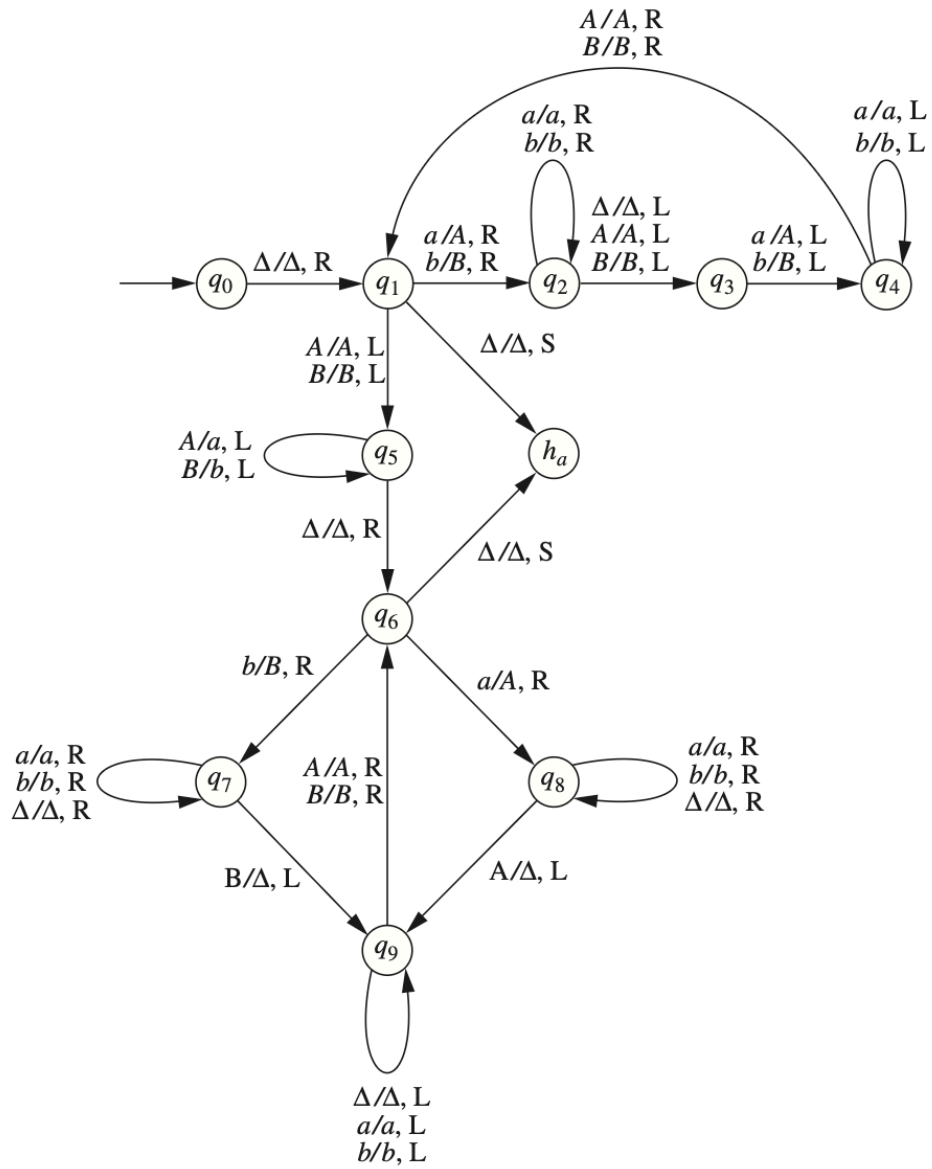


Figure 7.5 |

A Turing machine accepting $\{xx \mid x \in \{a, b\}^*\}$.

We trace the moves of the TM for three input strings: two that illustrate the two possible ways a string can be rejected, and one that is in L .

$q_0 \Delta aba$	$\vdash \Delta q_1 aba$	$\vdash \Delta A q_2 ba$	$\vdash^* \Delta A ba q_2 \Delta$	$\vdash \Delta A b q_3 a$
	$\vdash \Delta A q_4 b A$	$\vdash \Delta q_4 A b A$	$\vdash \Delta A q_1 b A$	$\vdash \Delta A B q_2 A$
	$\vdash \Delta A q_3 B A$	$\vdash \Delta A h_r B A$	(reject)	
$q_0 \Delta ab$	$\vdash \Delta q_1 ab$	$\vdash \Delta A q_2 b$	$\vdash \Delta A b q_2 \Delta$	$\vdash \Delta A q_3 b \Delta$
	$\vdash \Delta q_4 A B$	$\vdash \Delta A q_1 B$	$\vdash \Delta q_5 A B$	$\vdash q_5 \Delta a B$
	$\vdash \Delta q_6 a B$	$\vdash \Delta A q_8 B$	$\vdash \Delta A h_r B$	(reject)
$q_0 \Delta aa$	$\vdash \Delta q_1 aa$	$\vdash \Delta A q_2 a$	$\vdash \Delta A a q_2 \Delta$	$\vdash \Delta A q_3 a \Delta$
	$\vdash \Delta q_4 A A$	$\vdash \Delta A q_1 A$	$\vdash \Delta q_5 A A$	$\vdash q_5 \Delta a A$
	$\vdash \Delta q_6 a A$	$\vdash \Delta A q_8 A$	$\vdash \Delta q_9 A$	$\vdash \Delta A q_6 \Delta$
	$\vdash \Delta A h_a \Delta$	(accept)		

Q15. TM for accepting $L = \{a^i b a^j \mid 0 \leq i < j\}$

Soln

⇒ Turing Machines that compute Partial functions:

A computer program whose purpose is to produce an output string for every legal input string can be interpreted as computing a function from one set of strings to another. A Turing machine T with input alphabet Σ that does the same thing computes a function whose domain D is a subset of Σ^* . Things will be a little simpler if we say instead that T computes a partial function on Σ^* with domain D . For every input string x in the domain of the function, T carries out a computation that ends with the output string $f(x)$ on the tape. The definition below considers a partial function on $(\Sigma^*)^k$, a function of k string variables; when $k > 1$, the k strings all appear initially on the tape, separated by blanks. The value of the function is always assumed to be a single string, and most of the time we will consider only $k = 1$.

The most significant issue for a TM T computing a function f is what output strings it produces for input strings in the domain of f . However, what T does for other input strings is not completely insignificant, because we want to say that T computes the partial function f , not some other partial function with a larger domain. For this reason, we say that T should end up in the accepting state for inputs in the domain of f and not for any other inputs. In other words, if T computes a partial function with domain D , then in particular, T accepts D , irrespective of the output it produces.

[Definition]: A Turing Machine computing a function

Let $T = (Q, \Sigma, \Gamma, q_0, \delta)$ be a Turing machine, k a natural number, and f a partial function on $(\Sigma^*)^k$ with values in Γ^* . We say that T computes f if for every $(x_1, x_2, \dots, x_k)$ in the domain of f ,

$$q_0 \Delta x_1 \Delta x_2 \Delta \dots \Delta x_k \xrightarrow{*} \Delta f(x_1, x_2, \dots, x_k)$$

and no other input that is k -tuple of strings is accepted by T

A partial function $f: (\Sigma^*)^k \longrightarrow \Gamma^*$ is Turing-computable, or simply computable, if there is a TM that computes f .

→ Characteristic Function of a set:

For a language $L \subseteq \Sigma^*$, the characteristic function of L is the function $\chi_L: \Sigma^* \rightarrow \{0,1\}$ defined by:

$$\chi_L(x) = \begin{cases} 1 & \text{if } x \in L \\ 0 & \text{if } x \notin L \end{cases}$$

Here we think of 0 and 1 as alphabet symbols, rather than numbers, so that when we use a TM to compute χ_L , unary notation is not involved.

- Computing the function χ_L and accepting the language L are two

approaches to the question of whether an arbitrary string is in L , and Turing machines that carry out these computations are similar in some respects.

⇒ Combining Turing Machines:

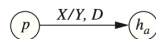
- A Turing machine represents an algorithm
- we can combine several TMs into a larger composite TM
- in the simplest case, if T_1 and T_2 are TMs, we can consider the composition:

$T_1 T_2$

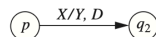
whose composition can be described approximately by saying
"first execute T_1 , then execute T_2 ."

stops.

If $T = (Q, \Sigma, \Gamma, q_0, \delta)$ is the composite TM $T_1 T_2$, we can obtain the state set Q (of nonhalting states) of T by taking the union of the state sets of T_1 and T_2 , except that the states of T_2 are relabeled if necessary so that the two sets don't overlap. The initial state of T is the initial state of T_1 . The transitions of T include all those of T_2 , and all those of T_1 that don't go to h_a . A transition



in T_1 becomes



in T , where q_2 is the initial state of T_2 . In other words, T begins in the initial state of T_1 and executes the moves of T_1 up to the point where T_1 would halt. If T_1 halts in the reject state, then so does T , but if T_1 halts in h_a , then at that point T_2 takes over, in its initial state. The moves that cause T to accept are precisely those that cause T_2 to accept.

We can usually avoid being too explicit about how the input alphabets and tape alphabets of T , T_1 , and T_2 are related. It may be, for example, that T_1 's job is to create an input configuration different from the one T_2 would normally have started with; or it may be that T_2 does not expect to be processing input at all, but only making some modification to the tape configuration in preparation for some other TM. It should be the case, at least, that any of T_1 's tape symbols that T_2 is likely to encounter should be in T_2 's input alphabet.

It may now be a little clearer why in certain types of computations we are careful to specify the final tape configuration. If T_1 is to be followed by another TM T_2 , T_1 must finish up so that the tape looks like what T_2 expects when it starts. For example, if T_1 and T_2 compute the functions f and g , respectively, from $\mathcal{N}$ to $\mathcal{N}$, the output configuration left by T_1 is a legitimate input configuration for T_2 , so that the output from input n is $g(f(n))$. In other words, $T_1 T_2$ computes the composite function $g \circ f$, defined by the formula $g \circ f(n) = g(f(n))$.

⇒ Multitape Turing Machine:

- a k -tape Turing machine has:

- k tapes
- k heads
- In one step, it can:
 - Read k symbols
 - Write k symbols
 - Move k heads independently

Theorem 7.26

For every 2-tape TM $T = (Q, \Sigma, \Gamma, q_0, \delta)$, there is an ordinary 1-tape TM $T_1 = (Q_1, \Sigma, \Gamma_1, q_1, \delta_1)$, with $\Gamma \subseteq \Gamma_1$, such that

1. For every $x \in \Sigma^*$, T accepts x if and only if T_1 accepts x , and T rejects x if and only if T_1 rejects x . (In particular, $L(T) = L(T_1)$.)
2. For every $x \in \Sigma^*$, if

$$(q_0, \underline{\Delta}x, \underline{\Delta}) \vdash_T^* (h_a, y\underline{a}z, u\underline{b}v)$$

for some strings $y, z, u, v \in (\Gamma \cup \{\Delta\})^*$ and symbols $a, b \in \Gamma \cup \{\Delta\}$, then

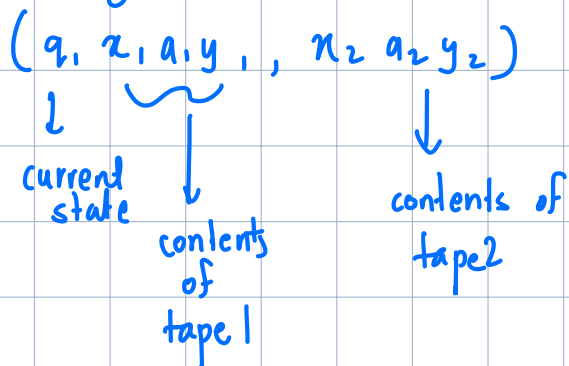
$$q_1 \Delta x \vdash_{T_1}^* y h_a a z$$

For 2-tape TM:

$$T = (Q, \Sigma, \Gamma, q_0, \delta)$$

$$\delta : Q \times (\Gamma \cup \{\Delta\})^2 \rightarrow (Q \cup \{h_a, h_r\}) \times (\Gamma \cup \{\Delta\})^2 \times \{L, R, S\}^2$$

Configuration:



[THEOREM]: For every 2-tape TM T , there exists a 1-tape TM T_1 , such that:

(i) Both accept exactly the same language

(ii) Both reject the same strings

(iii') Both produce the same output

$$L(T) = L(T_1)$$

Note: • Every language accepted by a multitape TM can be accepted by a single-tape TM

• Every function computed by a multitape TM can be computed by a single-tape TM.

⇒ Non Deterministic Turing Machine:

- Deterministic TMs have:

for each (state, symbol) $\rightarrow$ exactly one move

- A non-deterministic TM (NTM) T is defined the same way as an ordinary TM, except that for a state-symbol pair there might be more than one move.

if $T = (Q, \Sigma, \Gamma, q_0, \delta)$

and $(q, a) \in Q \times (\Gamma \cup \{\Delta\})$

then $\delta(q, a)$ is a finite subset, not an element, of $(Q \cup \{h, r\}) \times (\Gamma \cup \{\Delta\}) \times \{R, L, S\}$

Now, if,

p is a non-halting state, q is any state, a and b are tape symbols, and w, x, y, z are strings,

$$w p a x \vdash_T y q b z$$

or

$$w p a x \vdash_T^* y q b z$$

And,

$$q_0 \Delta x \vdash_T^* w h a y$$

↳

string x is accepted by T for some strings $w, y \in (\Gamma \cup \{\Delta\})^*$

Note:

- An NTM does not choose the correct move intelligently, it branches exploring all possibilities in parallel.
- A string x is accepted if at least one computation path accepts

Note: NTMs are great when you want to guess something then verify it

↳ GUESS + CHECK

- Example: Composite Numbers

Let L be the subset of $\{1\}^*$ of all strings whose length is a composite number. Since a prime number is a natural number 2 or higher whose only positive divisors are itself and 1, and element of L is a string of the form 1^n , where $n = p \times q$, for some integers p and q with $p, q \geq 2$

We need to guess p and q non-deterministically

- if n is composite:

- there exists some current guess
- that computation path exists

- if n is prime:

- no guess works
- no path accepts

Component	Role
NB	Move to next blank
PB	Move to previous blank
G_2	Generate ≥ 2 ones nondeterministically
M	Multiply
Equal	Compare results

9. Pipeline (MEMORIZE THIS STYLE)

CSS

Copy code

NB $\rightarrow$ G₂ $\rightarrow$ NB $\rightarrow$ G₂ $\rightarrow$ PB $\rightarrow$ M $\rightarrow$ PB $\rightarrow$ Equal

What this means in English:

1. Skip input
2. Guess `p`
3. Skip
4. Guess `q`
5. Go back
6. Compute `p × q`
7. Go back
8. Compare with original `n`

- Example: Prefix Language

$$P(L) = \{x \mid \exists y \text{ such that } xy \in L\}$$

x is a prefix of some string in L

we guess for y non-deterministically

NB $\rightarrow$ G $\rightarrow$ Delete $\rightarrow$ PB $\rightarrow$ T

$\Downarrow$

If :

- there exists some y such that $xy \in L$
- that branch accepts

If not:

- no branch accepts

A *nondeterministic* Turing machine (NTM) T is defined the same way as an ordinary TM, except that for a state-symbol pair there might be more than one move. This means that if $T = (Q, \Sigma, \Gamma, q_0, \delta)$, and $(q, a) \in Q \times (\Gamma \cup \{\Delta\})$, then $\delta(q, a)$ is a finite subset, not an element, of $(Q \cup \{h_a, h_r\}) \times (\Gamma \cup \{\Delta\}) \times \{R, L, S\}$.

We can use the same notation for configurations of an NTM as in Section 7.1; if p is a nonhalting state, q is any state, a and b are tape symbols, and w, x, y, z are strings,

$$wpax \vdash_T yqbz$$

means that T has at least one move in the first configuration that leads to the second, and

$$wpax \vdash_T^* yqbz$$

means that there is at least one sequence of zero or more moves that takes T from the first to the second. It is still correct to say that a string x is accepted by T if

$$q_0 \Delta x \vdash_T^* wh_a y$$

for some strings $w, y \in (\Gamma \cup \{\Delta\})^*$.

The idea of an NTM that produces output will be useful, particularly one that is a component in a larger machine. We will not consider the idea of computing a function using a nondeterministic Turing machine, however, because a TM that computes a function should produce no more than one output for a given input string.

There are many types of languages for which nondeterminism makes it easy to construct a TM accepting the language. We give two examples, and there are others in the exercises.

[THEOREM] : For every NTM T , there exists a DTM T_1 such that
 $L(T_1) = L(T)$



NTMs are not more powerful

Key Observation

An NTM computation:

- Is a tree
- Each path = sequence of choices

Strategy

Try all possible paths, one by one

17. Encoding Choices as Binary Strings

Assume:

- At most 2 choices per step
- Label them 0 and 1

Then:

- Each computation path = binary string
- Example:
 - 0101 → take move 0, then 1, then 0, then 1

18. Order of Testing (Canonical Order)

Test paths in this order:

Copy code

```
ε  
0, 1  
00, 01, 10, 11  
000, ...
```

This ensures:

- If a path exists, it will eventually be tested



20. Four-Tape Simulation (Intuition)

Tape	Purpose
Tape 1	Input (read-only)
Tape 2	Current binary choice string
Tape 3	Simulated work tape of NTM
Tape 4	Tracks rejection depth

Tape 4 ensures:

- DTM can reject if all paths fail within n steps

⇒ Universal Turing Machines:

- A universal TM is a TM that can simulate any other TM

A universal Turing machine is a TM T_u that works as follows:
It is assumed to receive an input string of the form $e(T)e(z)$,
where T is an arbitrary TM, z is a string over the input alphabet
of T , and e is an encoding function whose values are strings in
 $\{0,1\}^*$. The computation performed by T_u on this input string
satisfies 2 properties:

- ① T_u accepts the string $e(T)e(z)$ if and only if T accepts z
- ② If T accepts z and produces output y , then T_u produces $e(y)$

Note: This is the theoretical basis of:

- stored-program computers
- compilers
- interpreters

• virtual machines

- Encoding:

(i) Encoding symbols:

let the tape alphabet be:

$$\Gamma = \{a_1, a_2, \dots, a_k\}$$

Encoding rule:

$$a_i \mapsto 1^i$$

$$a_1 = \Delta (\text{blank}) \rightarrow 1$$

$$a_2 \rightarrow 11$$

$$a_3 \rightarrow 111$$

(b) Encoding states:

$$q_1 \rightarrow 1$$

$$q_2 \rightarrow 11$$

$$q_3 \rightarrow 111$$

(c) Encoding directions:

$$\text{Right (R)} \rightarrow 1$$

$$\text{left (L)} \rightarrow 11$$

$$\text{Stay (S)} \rightarrow 111$$

★★

(d) Encoding a Move:

A TM move is a 5 tuple:

$$\delta(p, \sigma) = (q, \tau, D)$$

This contains:

- (i) current state
- (ii) current symbol
- (iii) Next state
- (iv) symbol to write
- (v) head direction

- each component is encoded in unary, separated by 0s:

General form:

$$1^n(p) 0 1^m(\sigma) 0 1^n(q) 0 1^k(\tau) 0 1^n(D) 0$$

★ 0 acts like delimiter

(e) Encoding an Entire TM:

a TM has many moves:

$$m_1, m_2, \dots, m_k$$

we encode the full machine by concatenating all moves:

$$e(1) = e(m_1)0e(m_2)0 \dots 0e(m_k)0$$

so the TM becomes a binary string

Note : Why this encoding is safe:

- (1) No ambiguity
- (2) Decidable validity

Note: Decodability:

Another TM can:

- Read the encoding

- Recover:

- states

- symbols

- transition

no information is lost

→ Working of a Universal TM:

— A universal TM T_u simulates any TM T on any input z

— Tapes used:

① Tape 1 → Encoded TM + input : $e(1)e(z)$

② Tape 2 → Simulated work tape of T

③ Tape 3 → Encoded current state

- Simulation loop:

At each step:

- ① read current state from tape 3
- ② Read current symbol from tape 2
- ③ Scan tape 1 to find matching:
 $\delta(p, \sigma)$
- ④ Apply the rule:
 - Write symbol on tape 2
 - Move head on tape 2
 - Update state on Tape 3

3.1 Halting Behavior (VERY IMPORTANT)

Original TM T

Universal TM T_u

Accepts

Accepts + outputs result

Rejects

Rejects

Loops forever

Loops forever